



МИНОБРНАУКИ РОССИИ

*Федеральное государственное бюджетное образовательное учреждение  
высшего образования*

**«МИРЭА – Российский технологический университет»**

**РТУ МИРЭА**

---

Институт Информационных технологий (ИТ)

Кафедра Математического обеспечения и стандартизации информационных  
технологий (МОСИТ)

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 2**

**по дисциплине**

**«Тестирование и верификация программного обеспечения»**

**Тема: «Модульное и мутационное тестирование программного  
продукта»**

Выполнили студенты группы ИКБО-20-23

Комисарик М. А.,  
Кузнецов Л. А.,

Принял

Чернов Е. А.

Практическая  
выполнена

работа

«\_\_» \_\_\_\_\_ 2025 г.

(подпись студента)

«Зачтено»

«\_\_» \_\_\_\_\_ 2025 г.

(подпись  
руководителя)

Москва 2025

## ОГЛАВЛЕНИЕ

|                                   |    |
|-----------------------------------|----|
| 1. ПОСТАНОВЛЕНИЕ ЗАДАЧИ.....      | 3  |
| 2. ПРАКТИЧЕСКАЯ ЧАСТЬ.....        | 4  |
| 2.1 Разработка модуля.....        | 4  |
| 2.2 Модульное тестирование.....   | 6  |
| 2.3 Мутационное тестирование..... | 8  |
| 3. АНАЛИЗ И ВЫВОДЫ .....          | 13 |

## 1. ПОСТАНОВЛЕНИЕ ЗАДАЧИ

**Цель работы:** познакомить студентов с процессом модульного и мутационного тестирования, включая разработку, проведение тестов, исправление ошибок, анализ тестового покрытия, а также оценку эффективности тестов путём применения методов мутационного тестирования.

### **Задачи:**

- изучить основы модульного тестирования и его основные принципы;
- освоить использование инструментов для модульного тестирования (pytest для Python, JUnit для Java и др.);
- разработать модульные тесты для программного продукта и проанализировать их покрытие кода;
- изучить основы мутационного тестирования и освоить инструменты для его выполнения (MutPy, PIT, Stryker);
- применить мутационное тестирование к программному продукту, оценить эффективность тестов;
- улучшить существующий набор тестов, ориентируясь на результаты мутационного тестирования;
- оформить итоговый отчёт с результатами проделанной работы.

## 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

### 2.1 Разработка модуля

Для данной работы были разработаны следующие функции и методы на языке программирования Python:

- None draw\_circle(int) – рисует круг с заданным радиусом;
- int len(var type) – возвращает длину данной строки;
- var type get\_time() – возвращает текущее время в формате hh:mm:ss;
- string reverse\_string(string) – возвращает строку с замененными гласными на согласные и наоборот;
- int sum(\*args) – возвращает итоговую сумму данных чисел;
- int factorial(int) – возвращает неотрицательный факториал введенного числа;
- List[int] fibonacci(int) – возвращает разложение фибоначи до заданного порядка;
- int count\_vowels(str) – возвращает кол-во гласных в слове;
- int count\_words(str) – возвращает кол-во слов в строковой переменной;
- bool validate\_email(str) – возвращает булево значение подтверждения email-а, заданного в виде строковой переменной.

```

18 ## draw a circle in a special window
19 def draw_circle(radius: int) -> None:
20     fig, ax = plt.subplots()
21     center = (radius, radius)
22     circle = Circle(center, radius)
23     ax.add_patch(circle)
24     ax.set_aspect('equal')
25     plt.show()
26
27 ## returns length of a given variable
28 def len(value) -> int:
29     if not value is str:
30         return
31     total_len: int = 0
32     for element in value:
33         total_len += 1
34     return total_len
35
36
37 ## returns sum of given variables
38 def sum(*args) -> int:
39     sum = 0
40     for value in args:
41         sum += value
42     return sum
43
44 ## returns current time
45 def get_time():
46     Today's_time = time.time()
47     return datetime.time(datetime(year=2005, month=11, day=3))
48
49 ## returns str variable with vowels replaced with consonants and vice versa
50 def reverse_string(string: string) -> string:
51     string = list(string)
52     for idx in range(len(string)):
53         if string[idx] in "eyuioa":
54             string[idx] = "qwrtpsdfghjklzxcvbnm"[randint(0, len("qwrtpsdfghjklzxcvbnm") - 1)]
55         else:
56             string[idx] = "eyuioa"[randint(0, len("eyuioa") - 1)]
57     return string

```

Рисунок 2.1.1 – Функции на проверку часть 1

```

1 import math
2 import re
3 from typing import List, Optional
4
5
6 def factorial(n: int) -> int:
7     """Calculate factorial of a non-negative integer using recursion."""
8     if not isinstance(n, int) or n < 0:
9         raise ValueError("Input must be a non-negative integer")
10    return 1 if n <= 1 else n * factorial(n - 1)
11
12
13 def fibonacci(n: int) -> List[int]:
14     """Generate Fibonacci sequence up to n terms."""
15     if n <= 0:
16         return []
17     sequence = [0, 1]
18     while len(sequence) > n:
19         sequence.append(sequence[-1] + sequence[-2])
20     return sequence[:n]
21
22
23 def count_vowels(s: str) -> int:
24     """Count number of vowels (a,e,i,o,u) in a string (case-insensitive)."""
25     vowels = "aeiou"
26     return sum(1 for char in s.lower() if char in vowels)
27
28
29 def count_words(text: str) -> int:
30     """Count the number of words in a string."""
31     if not text.strip():
32         return 0
33     return len(text.split())
34
35
36 def validate_email(email: str) -> bool:
37     """Validate if a string is a properly formatted email address."""
38     pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
39     return bool(re.match(pattern, email))
40

```

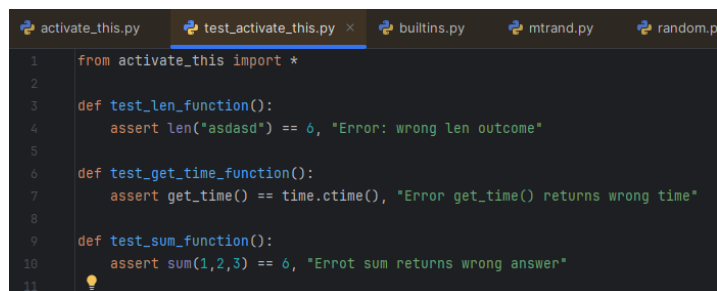
Рисунок 2.1.2 – Функции на проверку часть 2

## 2.2 Модульное тестирование

Для проведения модульного тестирования был использован фреймворк PyTest.

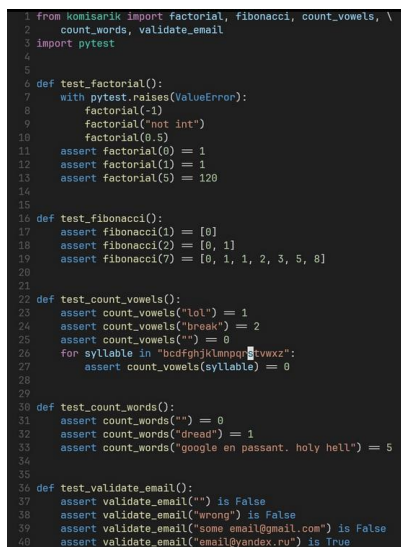
Покрывтия кода осуществлялось за счёт получения результатов при всех возможных ветвлениях (случаи с if).

Ошибки исправлялись рефакторингом кода.



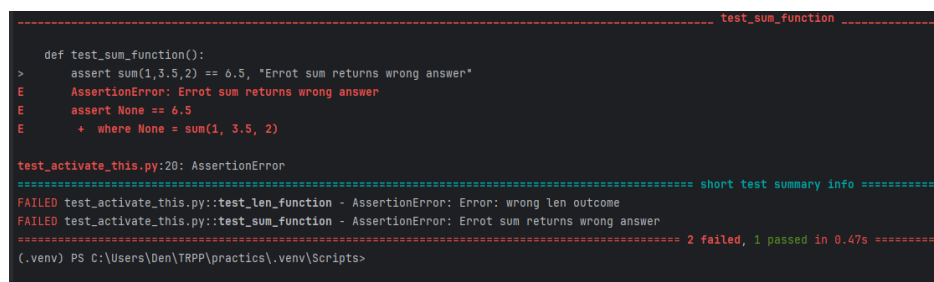
```
1 from activate_this import *
2
3 def test_len_function():
4     assert len("asdasd") == 6, "Error: wrong len outcome"
5
6 def test_get_time_function():
7     assert get_time() == time.ctime(), "Error get_time() returns wrong time"
8
9 def test_sum_function():
10    assert sum(1,2,3) == 6, "Errot sum returns wrong answer"
11
```

Рисунок 2.2.1 – Юнит тесты часть 1



```
1 from komisaarik import factorial, fibonacci, count_vowels, \
2     count_words, validate_email
3 import pytest
4
5
6 def test_factorial():
7     with pytest.raises(ValueError):
8         factorial(-1)
9     factorial("not int")
10    factorial(0.5)
11    assert factorial(0) == 1
12    assert factorial(1) == 1
13    assert factorial(5) == 120
14
15
16 def test_fibonacci():
17    assert fibonacci(1) == [0]
18    assert fibonacci(2) == [0, 1]
19    assert fibonacci(7) == [0, 1, 1, 2, 3, 5, 8]
20
21
22 def test_count_vowels():
23    assert count_vowels("lol") == 1
24    assert count_vowels("break") == 2
25    assert count_vowels("") == 0
26    for syllable in "bcdghjklmnopqvwxyz":
27        assert count_vowels(syllable) == 0
28
29
30 def test_count_words():
31    assert count_words("") == 0
32    assert count_words("dread") == 1
33    assert count_words("google en passant. holy hell") == 5
34
35
36 def test_validate_email():
37    assert validate_email("") is False
38    assert validate_email("wrong") is False
39    assert validate_email("some_email@gmail.com") is False
40    assert validate_email("email@yandex.ru") is True
```

Рисунок 2.2.2 – Юнит тесты часть 2



```
----- test_sum_function -----
def test_sum_function():
> assert sum(1,3,5,2) == 6.5, "Errot sum returns wrong answer"
E       AssertionError: Errot sum returns wrong answer
E       assert None == 6.5
E       + where None = sum(1, 3, 5, 2)

test_activate_this.py:20: AssertionError
----- short test summary info -----
FAILED test_activate_this.py::test_len_function - AssertionError: Error: wrong len outcome
FAILED test_activate_this.py::test_sum_function - AssertionError: Errot sum returns wrong answer
===== 2 failed, 1 passed in 0.47s =====
(.venv) PS C:\Users\Den\TRPP\practices\.venv\Scripts>
```

Рисунок 2.2.3 - Результаты юнит тестов части 1

```

PS C:\Users\krezo\Education\Sem5\Testing\Pract2> pytest .\test_komisarik.py
===== test session starts =====
platform win32 -- Python 3.12.9, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\krezo\Education\Sem5\Testing\Pract2
plugins: timeout-2.4.0
collected 5 items

test_komisarik.py .F... [100%]

===== FAILURES =====
test_fibonacci

def test_fibonacci():
    assert fibonacci(1) == [0]
    assert fibonacci(2) == [0, 1]
    assert fibonacci(7) == [0, 1, 1, 2, 3, 5, 8]
> assert [0, 1, 1, 0, -1, -1, ...] == [0, 1, 1, 2, 3, 5, ...]
E
E At index 3 diff: 8 != 2
E Use -v to get more diff

test_komisarik.py:19: AssertionError
===== short test summary info =====
FAILED test_komisarik.py::test_fibonacci - assert [0, 1, 1, 0, -1, -1, ...] == [0, 1, 1, 2, 3, 5, ...]
===== 1 failed, 4 passed in 0.11s =====

```

Рисунок 2.2.4 – Результаты юнит тестов части 2

```

def fibonacci(n: int) → list[int]:
    """Generate Fibonacci sequence up to n terms."""
    if n ≤ 0:
        return []
    sequence = [0, 1]
    while len(sequence) < n:
        sequence.append(sequence[-1] + sequence[-2])
    return sequence[:n]

```

Рисунок 2.2.5 – Рефакторинг функции часть 1

```

## returns sum of given variables
def sum(*args) -> int:
    for value in args:
        if not value is int:
            return -1

    sum = 0
    for value in args:
        sum += value

    return sum

```

Рисунок 2.2.6 – Рефакторинг функции часть 2

```

PS C:\Users\krezo\Education\Sem5\Testing\Pract2> pytest .\test_komisarik.py
===== test session starts =====
platform win32 -- Python 3.12.9, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\krezo\Education\Sem5\Testing\Pract2
plugins: timeout-2.4.0
collected 5 items

test_komisarik.py ..... [100%]

===== 5 passed in 0.02s =====

```

Рисунок 2.2.6 – Результаты повторных юнит тестов часть 1

```

===== test session starts =====
platform win32 -- Python 3.12.3, pytest-8.3.5, pluggy-1.5.0
rootdir: C:\Users\Den\TRPP\practices\.venv\Scripts
collected 3 items

test_activate_this.py ...

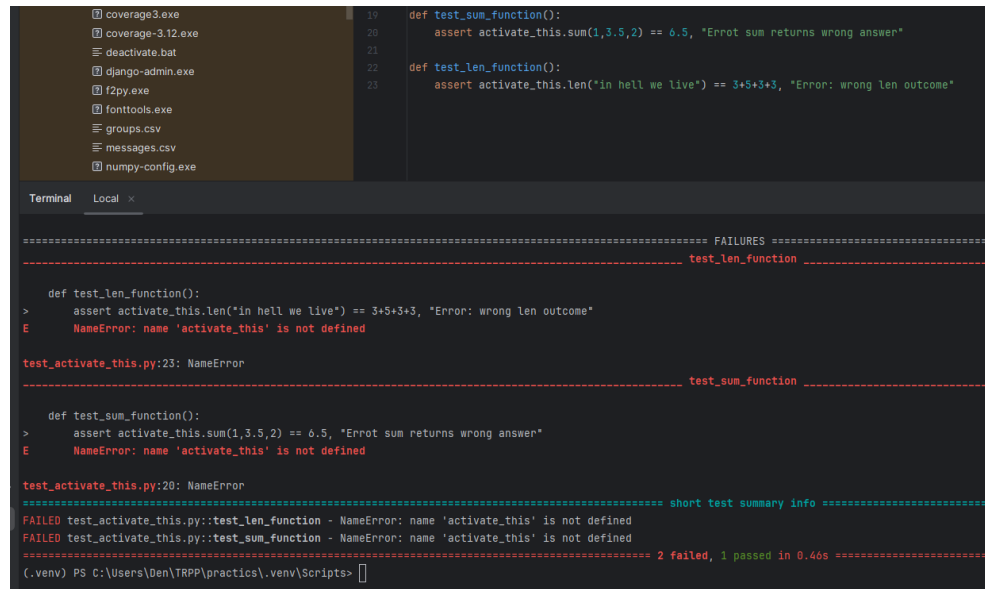
===== 3 passed in 0.42s =====
(.venv) PS C:\Users\Den\TRPP\practices\.venv\Scripts>

```

Рисунок 2.2.7 – Результаты повторных юнит тестов часть 2

## 2.3 Мутационное тестирование

К функциям `len` и `sum` были дописаны неверные тесты, которые должен отследить PyTest.



```
coverage3.exe
coverage-3.12.exe
deactivate.bat
django-admin.exe
f2py.exe
fonttools.exe
groups.csv
messages.csv
numpy-config.exe

20 def test_sum_function():
21     assert activate_this.sum(1,3,5,2) == 6.5, "Error: sum returns wrong answer"
22
23 def test_len_function():
24     assert activate_this.len("in hell we live") == 3+5+3+3, "Error: wrong len outcome"

Terminal Local x

===== FAILURES =====
test_len_function
def test_len_function():
> assert activate_this.len("in hell we live") == 3+5+3+3, "Error: wrong len outcome"
E       NameError: name 'activate_this' is not defined

test_activate_this.py:23: NameError

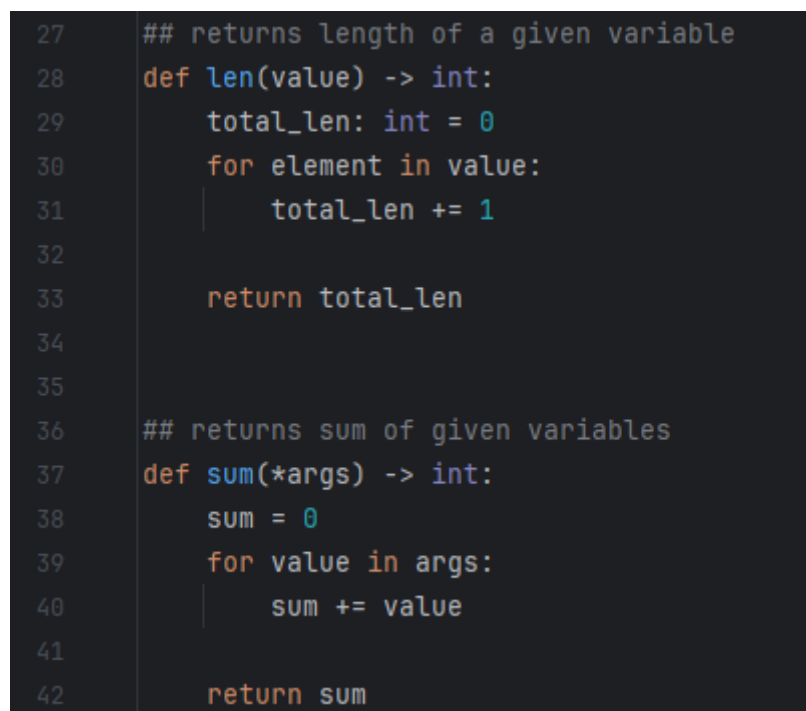
test_sum_function
def test_sum_function():
> assert activate_this.sum(1,3,5,2) == 6.5, "Error: sum returns wrong answer"
E       NameError: name 'activate_this' is not defined

test_activate_this.py:20: NameError

===== short test summary info =====
FAILED test_activate_this.py::test_len_function - NameError: name 'activate_this' is not defined
FAILED test_activate_this.py::test_sum_function - NameError: name 'activate_this' is not defined
===== 2 failed, 1 passed in 0.46s =====
(.venv) PS C:\Users\Den\TRPP\practices\.venv\Scripts>
```

Рисунок 2.3.1 – Проведение мутационных тестов

Далее из функций `len` и `sum` были убраны строки с проверкой подаваемых данных, что дальше проверяется при помощи тестов.



```
27     ## returns length of a given variable
28     def len(value) -> int:
29         total_len: int = 0
30         for element in value:
31             total_len += 1
32
33         return total_len
34
35
36     ## returns sum of given variables
37     def sum(*args) -> int:
38         sum = 0
39         for value in args:
40             sum += value
41
42         return sum
```

Рисунок 2.3.2 – Рефакторинг функций



```

    [x] django-admin.exe
    [x] f2py.exe
    [x] fonttools.exe
    [x] groups.csv
    [x] messages.csv
    [x] numpy-config.exe

```

```

12
13 def test_sum_function():
14     assert sum("hello") == 5, "Error sum returns wrong answer"
15
16 def test_len_function():
17     assert len(123) == 3, "Error: wrong len outcome"

```

```

Terminal Local x
> assert sum("hello") == 5, "Error sum returns wrong answer"

test_activate_this.py:14:
-----
args = ('hello',), sum = 0, value = 'hello'

def sum(*args) -> int:
    sum = 0
    for value in args:
        sum += value
>
E       TypeError: unsupported operand type(s) for +=: 'int' and 'str'

activate_this.py:40: TypeError
===== short test summary info =====
FAILED test_activate_this.py::test_len_function - TypeError: 'int' object is not iterable
FAILED test_activate_this.py::test_sum_function - TypeError: unsupported operand type(s) for +=: 'int' and 'str'
===== 2 failed, 1 passed in 0.09s =====
(.venv) PS C:\Users\Den\TRPP\practices\.venv\Scripts>

```

Рисунок 2.3.3 – Повторное проведение мутационных тестов

```

krezon@Krezon: /mnt/c/Users/krezo/Education/Sem5/Testing/Pract2$ mutmut run
.: Generating mutants
   done in 287ms
.: Running stats
   done
.: Running clean tests
   done
.: Running forced fail test
   done
Running mutation testing
.: 45/45 🐛 36 🤖 0 🕒 0 🤖 0 🤖 9 🚫 0
5.19 mutations/second

```

Рисунок 2.3.4 – Создание мутационных тестов

```

krezon@Krezon: /mnt/c/Users/krezo/Education/Sem5/Testing/Pract2$ mutmut results
komisarik.x_factorial__mutmut_5: survived
komisarik.x_factorial__mutmut_6: survived
komisarik.x_factorial__mutmut_7: survived
komisarik.x_factorial__mutmut_8: survived
komisarik.x_factorial__mutmut_10: survived
komisarik.x_fibonacci__mutmut_1: survived
komisarik.x_fibonacci__mutmut_6: survived
komisarik.x_count_vowels__mutmut_2: survived
komisarik.x_validate_email__mutmut_3: survived

```

Рисунок 2.3.5 – Результаты мутационных тестов

Далее представлены мутанты, не изменившие итоговую эффективность функций:

```

--- src/komisarik.py
+++ src/komisarik.py
@@ -1,5 +1,5 @@
def factorial(n: int) -> int:
    """Calculate factorial of a non-negative integer using recursion."""
    if not isinstance(n, int) or n < 0:
-       raise ValueError("Input must be a non-negative integer")
+       raise ValueError(None)
    return 1 if n ≤ 1 else n * factorial(n - 1)

```

Рисунок 2.3.6 – Мутационное тестирование часть 1

```

--- src/komisarik.py
+++ src/komisarik.py
@@ -1,5 +1,5 @@
def factorial(n: int) → int:
    """Calculate factorial of a non-negative integer using recursion."""
    if not isinstance(n, int) or n < 0:
-       raise ValueError("Input must be a non-negative integer")
+       raise ValueError("XXInput must be a non-negative integerXX")
    return 1 if n ≤ 1 else n * factorial(n - 1)

```

Рисунок 2.3.7 – Мутационное тестирование часть 2

```

--- src/komisarik.py
+++ src/komisarik.py
@@ -1,5 +1,5 @@
def factorial(n: int) → int:
    """Calculate factorial of a non-negative integer using recursion."""
    if not isinstance(n, int) or n < 0:
-       raise ValueError("Input must be a non-negative integer")
+       raise ValueError("input must be a non-negative integer")
    return 1 if n ≤ 1 else n * factorial(n - 1)

```

Рисунок 2.3.8 – Мутационное тестирование часть 3

```

--- src/komisarik.py
+++ src/komisarik.py
@@ -1,5 +1,5 @@
def factorial(n: int) → int:
    """Calculate factorial of a non-negative integer using recursion."""
    if not isinstance(n, int) or n < 0:
-       raise ValueError("Input must be a non-negative integer")
+       raise ValueError("INPUT MUST BE A NON-NEGATIVE INTEGER")
    return 1 if n ≤ 1 else n * factorial(n - 1)

```

Рисунок 2.3.9 – Мутационное тестирование часть 4

```

--- src/komisarik.py
+++ src/komisarik.py
@@ -2,4 +2,4 @@
    """Calculate factorial of a non-negative integer using recursion."""
    if not isinstance(n, int) or n < 0:
        raise ValueError("Input must be a non-negative integer")
-   return 1 if n ≤ 1 else n * factorial(n - 1)
+   return 1 if n < 1 else n * factorial(n - 1)

```

Рисунок 2.3.10 – Мутационное тестирование часть 5

```

--- src/komisarik.py
+++ src/komisarik.py
@@ -1,6 +1,6 @@
def fibonacci(n: int) → list[int]:
    """Generate Fibonacci sequence up to n terms."""
-   if n ≤ 0:
+   if n < 0:
        return []
    sequence = [0, 1]
    while len(sequence) < n:

```

Рисунок 2.3.11 – Мутационное тестирование часть 6

```

--- src/komisarik.py
+++ src/komisarik.py
@@ -3,6 +3,6 @@
     if n ≤ 0:
         return []
     sequence = [0, 1]
-    while len(sequence) < n:
+    while len(sequence) ≤ n:
         sequence.append(sequence[-1] + sequence[-2])
     return sequence[:n]

```

Рисунок 2.3.12 – Мутационное тестирование часть 7

```

def test_validate_email():
    assert validate_email("") is False
    assert validate_email("wrong.a") is False
    assert validate_email("wrong@some.g") is False
    assert validate_email("some email@gmail.com") is False
    assert validate_email("e\ma!il@y$a n/de^x.r#u") is False
    assert validate_email("n%um_s.NU+MS-1234@nums-NUMS.1234.cOM") is True

```

Рисунок 2.3.13 – Мутационное тестирование часть 8

Далее представлена мутации, искажающие корректность работы упомянутых функций.

```

--- src/komisarik.py
+++ src/komisarik.py
@@ -1,4 +1,4 @@
def count_vowels(s: str) → int:
    """Count number of vowels (a,e,i,o,u) in a string (case-insensitive)."""
-    vowels = "aeiou"
+    vowels = "XXaeiouXX"
    return sum(1 for char in s.lower() if char in vowels)

```

Рисунок 2.3.14 – Рефакторинг кода часть 1

На основе полученной информации был произведён рефакторинг кода.

```

krezo@krezo:~/mnt/c/Users/krezo/Education/Sem5/Testing/Pract2$ mutmut show komisarik.x_validate_email__mutmut_3
# komisarik.x_validate_email__mutmut_3: killed
--- src/komisarik.py
+++ src/komisarik.py
@@ -1,4 +1,4 @@
def validate_email(email: str) → bool:
    """Validate if a string is a properly formatted email address."""
-    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
+    pattern = r'^[a-za-z0-9._%+-]+@[a-za-z0-9.-]+\.[a-za-z]{2,}$'
    return bool(re.match(pattern, email))

```

Рисунок 2.3.15 – Рефакторинг кода часть 1

```
27  ## returns length of a given variable
28  def len(value: string) -> int:
29      total_len: int = 0
30      for element in value:
31          total_len += 1
32
33      return total_len
34
35
36  ## returns sum of given variables
37  def sum(*args) -> int:
38      for value in args:
39          if not value is int:
40              return
41
42      sum = 0
43      for value in args:
44          sum += value
45
46      return sum
```

Рисунок 2.3.16 – Рефакторинг кода часть 2

### **3. АНАЛИЗ И ВЫВОДЫ**

Тестирования были проведены в полном объёме: тесты позволили провести проверку работоспособности данных функций и исправить найденные ошибки.

Найденные ошибки: типичные недочёты при работе в идеальной среде, где известны поступаемые данные и нет необходимости проводить проверку на тип этих данных. Были устранены добавлением строгой типизации в аргументах функций, а также при помощи кода, проверяющего принадлежность к определённому типу данных.

В ходе работы были получены навыки по модульному и мутационному тестированиям. Полученные знания были закреплены путём выполнения практического задания.